# IMCA4C03
# SYSTEM SOFTWARE

## Module 2

INTRODUCTION - System software and machine architecture - The Simplified Instructional Computer (SIC) Machine architecture - Data and instruction formats - addressing modes - Instruction sets - I/O and programming

# INTRODUCTION

- System software and machine architecture
- The Simplified Instructional Computer (SIC)
- Machine architecture
- Data and instruction formats
- Addressing modes
- Instruction sets
- I/O and programming

# SOFTWARE - ITS CLASSIFICATION

- Software is set of instructions or programs written to carry out certain task on digital computers.
- Classified into system software and application software.
- System software consists of a variety of programs that support the operation of a computer.
- Application software focuses on an application or problem to be solved.

# SYSTEM SOFTWARE

- The subject introduces the design and implementation of system software.
- System software consists of a variety of programs that support the operation of a computer.

# SYSTEM SOFTWARE

Examples of system software are:

- OS
- Compiler
- Assembler
- Macro Processor
- Loader or Linker
- Debugger
- Text Editor

# System software vs Application software

| System software | Application software |
|---|---|
| It is the kind of software which is the interface between the application software and system | It is the kind of software which runs according to the user. |
| Collection of programs that help the user to interact with hardware components efficiently. | Collection of programs written for a specific application. i.e. It is used by users to perform any specific task they want. |
| This kind of software is written inlow-levell language | This kind of software is written in high level language. |
| System software control and manage the hardware and hence system software directly interact with hardware | Application software uses the service of the system software to interact with hardware components. So, application software will not interact with hardware directly. |

# System software vs Application software

| System software | Application software |
|---|---|
| To write system software the programmer needs to understand the architecture and hardware details and hence system software are machine dependent | To write the application software the programmer need not worry about the architecture and hardware details and hence application software are machine independent |
| Programmer should be more familiar with architecture, instruction formats, addressing modes and so on | Programmer should be more familiar with programming languages, data structures and clear knowledge of the problem domain |
| Development of system software is complex task | Development of application software is relatively easier |
| Examples: compiler, assembler, operating system | Examples: ticket reservation, banking software, word processor, media player |
| This can run independently. | This can't run independently. They need the presence of system software. |

# THIS CHAPTER GIVES YOU…

- System Software & Machine Architecture
- The Simplified Instructional Computer SIC and SIC/XE

# SYSTEM SOFTWARE & MACHINE ARCHITECTURE

Machine Dependent

Machine Independent

## MACHINE INDEPENDENT

- There are aspects of system software that do not directly depend upon the type of computing system
  - general design and logic of an assembler
  - general design and logic of a compiler
  - code optimization techniques

## MACHINE DEPENDENT

- Assembler - translates mnemonic instructions into machine code
- Compilers - must generate machine language code

# THE SIMPLIFIED INSTRUCTIONAL COMPUTER (SIC)

SIC is a hypothetical computer that includes the hardware features most often found on real machines

Two versions of SIC

- standard model (SIC)
- extension version (SIC/XE)

  *(extra equipment or extra expensive)*

# SIC MACHINE ARCHITECTURE

SIC Machine architecture depends on the following features:

- Memory
- Registers
- Data Formats
- Instruction Formats
- Addressing Modes
- Instruction Set
- Input and Output

## 1. MEMORY

- Memory consists of 8-bit bytes
- 3 consecutive bytes form a word (24 bits)
- All locations are addressed by the location of their lowest numbered byte.
- A total of 32,768 ($2^{15}$) bytes in the computer memory.

## 2. REGISTERS

# Five registers - each 24 bits in length

| Mnemonic | Number | Use |
|---|---|---|
| A | 0 | Accumulator; used for arithmetic operations. |
| X | 1 | Index register; used for addressing |
| L | 2 | Linkage register; contains the return address whenever control transferred to subroutine. |
| PC | 8 | Program Counter; contains the address of the next instruction to be executed. |
| SW | 9 | Status Word, including Condition code such as <, >, = = |

## 3. DATA FORMATS

- **Integers** are stored as 24-bit binary numbers.
- 2's complement representation is used for negative values.

- **Characters** are stored using 8-bit ASCII codes.

- **No floating-point** hardware on the standard version of SIC.

# INSTRUCTION FORMATS & ADDRESSING MODES

## 4. Instruction Formats

- All machine instructions on the standard version of SIC have 24-bit format as shown below.

| | | |
|---|---|---|
| **Opcode (8)** | **x** | **Address (15)** |

x is used to indicate indexed addressing mode.

## 5. Addressing Modes

- There are two addressing modes available, which are as shown in the below table.
- Parentheses are used to indicate the contents of a register or a memory location.

| Mode | Indication | Target address calculation |
|---|---|---|
| Direct | x=0 | TA=address |
| Indexed | x=1 | TA=address+(X) |

(X) - represents contents of index register X.

## 6. INSTRUCTION SET

1. Load and store registers - LDA, LDX, STA, STX, etc.

- E.g. LDA #3 load 3 into Accumulator
- STA TBL means the value present in A will be stored into TBL

2. Integer arithmetic operations - ADD, SUB, MUL, DIV, etc.

- *All arithmetic operations involve register A and a word in memory, with the result being left in the register.*

## INSTRUCTION SET

3. Comparison instruction:

- *COMP - compares the value in register A with a word in memory, this instruction sets a condition code (CC) to indicate the result*

4. Conditional jump instructions:

- JLT, JEQ, JGT
- *These instructions test the setting of CC and jump accordingly*

## INSTRUCTION SET

5. Subroutine linkage: JSUB, RSUB

- *JSUB jumps to the subroutine, placing the return address in register L*
- *RSUB returns by jumping to the address contained in register L*

## INSTRUCTION SET

6. I/O (Input and Output are performed by transferring ***1 byte at a time*** to/from the rightmost 8 bits of register A. )

- The Test Device instruction (TD) tests whether the addressed device is ready to send or receive a byte of data. (either < (if free) or = (if busy))
- Read Data (RD)
- Write Data (WD)

## DATA MOVEMENT

**3-byte word:**

- LDA, STA, LDL, STL, LDX, STX

*A- Accumulator, L – Linkage Register, X – Index Register*

**1-byte:**

- LDCH, STCH *[LDCH instruction is used to load the data byte from memory into register A or accumulator.]*
- No memory-memory move instruction

## STORAGE DEFINITION

Directives :

WORD → ONE-WORD CONSTANT
RESW → ONE-WORD VARIABLE
BYTE → ONE-BYTE CONSTANT
RESB → ONE-BYTE VARIABLE

# SIC

Note:

SIC is a hypothetical machine (used for teaching purposes).

Memory is typically addressed using absolute addressing in SIC.

Instructions are generally executed sequentially unless modified by a jump or branch instruction.